

AI Software Engineering Agent: **A Research-Oriented Study of LLM-Based Repository-Level Software Engineering and Autonomous Software Repair**

Abstract

Large Language Models have demonstrated strong capabilities in code generation and natural language understanding, but using them for realistic software engineering tasks remains challenging. Software development often requires understanding multiple files, identifying dependencies, modifying existing code, running tests, and responding to execution feedback. A simple prompt-based code generation system is therefore insufficient for repository-level tasks.

In this project, I developed an AI Software Engineering Agent that explores how an LLM can be integrated into an iterative software development workflow. The system accepts a software engineering task, retrieves relevant repository context, uses an LLM to reason about the problem and generate code changes, interacts with software-engineering tools, and executes tests to verify the generated solution. Test results are used as feedback so that the agent can refine its solution when the initial modification fails.

The design of the system is motivated by research in Transformer-based language models, agentic reasoning and tool use, repository-level code generation, retrieval-augmented generation, automated program repair, and software engineering benchmarks such as SWE-bench. The project therefore serves both as a practical implementation and as an exploration of the challenges involved in developing reliable LLM-based software engineering agents.

1. Introduction

Large Language Models (LLMs) have significantly changed the way artificial intelligence can be applied to software development. Modern language models are capable of generating code, explaining programs, identifying potential bugs, writing documentation, and assisting developers with programming tasks. However, real-world software engineering is more complex than generating an isolated code snippet.

A software repository typically contains multiple files, modules, dependencies, configuration files, documentation, and test cases. A change made to one file can affect several other components of the system. Therefore, solving a software engineering problem requires repository-level understanding rather than only understanding the code contained in a single prompt.

Another important limitation is that generated code cannot always be assumed to be correct. An LLM may produce code that appears logically reasonable but fails during execution or introduces regressions. This makes testing and external feedback important components of an AI-based software engineering system.

These challenges motivated me to explore an agent-based approach to software engineering. Instead of using an LLM as a one-shot code generator, I designed a system in which the model can retrieve relevant repository information, reason about a task, interact with software-engineering tools, generate modifications, execute tests, observe the results, and iteratively improve its solution.

The project is therefore focused on exploring the transition from simple LLM-based code generation toward an autonomous and feedback-driven software engineering workflow.

2. Problem Statement

Traditional LLM-based coding systems generally operate by receiving a prompt and generating a response. Although this approach is useful for small programming tasks, it does not fully address the requirements of real-world software engineering.

The major challenges considered in this project are:

1. **Repository-level context:** Relevant information about a software task may be distributed across multiple files and modules.
2. **Context selection:** Providing an entire repository to an LLM may be inefficient or exceed the model's context limitations. The system therefore needs to identify and retrieve the most relevant information.
3. **Reliable code generation:** Generated code may contain logical errors, incorrect assumptions, or inconsistencies with the existing repository.
4. **Tool interaction:** Software engineering requires actions such as searching files, reading source code, modifying files, and executing tests.
5. **Verification:** A generated solution should be evaluated through actual execution rather than being accepted solely because it appears correct.
6. **Iterative correction:** When a generated modification fails, the system should be capable of using the failure information to improve the solution.

The problem addressed by this project is therefore:

How can an LLM-based agent be designed to understand repository-level context, perform software engineering actions, generate code modifications, and use automated testing as feedback to iteratively improve its solutions?

3. Objectives

The primary objectives of this project are:

- To explore the application of Large Language Models to software engineering tasks.
- To develop an agent capable of interacting with a software repository.
- To retrieve relevant repository context before generating code.
- To incorporate tool calling into the software engineering workflow.
- To enable the agent to generate and apply code modifications.
- To execute automated tests on generated changes.
- To use test results as feedback for iterative refinement.
- To evaluate the performance of the system using software engineering tasks and benchmark-style evaluation.
- To identify the limitations and reliability challenges of LLM-based coding agents.
- To understand how concepts from recent AI-for-Software-Engineering research can be translated into a working system.

4. Motivation

The motivation for this project comes from the increasing use of Large Language Models in software development. Tools based on LLMs can generate code and provide programming assistance, but real-world software development involves much more than generating isolated code snippets.

A developer working on an existing repository must first understand the structure of the project, identify relevant files, understand dependencies, modify existing implementations, and verify that the changes do not introduce new problems. These activities require interaction with the software environment and continuous feedback.

I therefore wanted to explore whether an LLM could be placed inside an agentic workflow where it is not limited to answering programming questions but can also interact with a repository and use external feedback to improve its decisions.

The project was also motivated by recent research in AI for Software Engineering (AI4SE), particularly research involving repository-level code generation, autonomous coding agents, automated program repair, retrieval-augmented generation, and software engineering benchmarks.

Rather than treating the LLM as the complete solution, this project treats the LLM as one component of a larger software engineering system.

5. Background

5.1 Large Language Models

Large Language Models are neural-network-based models trained on large collections of text and code. They learn statistical patterns and relationships within the training data and can generate text or code based on a given context.

In software engineering, LLMs can be used for code generation, code explanation, debugging, documentation, test generation, and program repair.

However, an LLM's generated response is not guaranteed to be correct. Therefore, additional context and external verification are important when applying LLMs to software engineering.

5.2 Transformer Architecture

Modern LLMs are primarily based on Transformer architectures. Transformers use attention mechanisms to model relationships between different elements of an input sequence.

Understanding the Transformer architecture was important for this project because it provides the foundation for understanding how modern LLM-based coding systems operate.

The Transformer concept also helped me understand why models can process relationships between different pieces of textual and programming context.

5.3 Embeddings

An embedding represents text or code as a numerical vector. Similar pieces of information can have similar representations in the embedding space.

For repository-level software engineering, embeddings can be used to represent code chunks, documentation, functions, or other repository information. These representations can then be used to retrieve information that is semantically relevant to a user's task.

5.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines information retrieval with language-model generation.

Instead of directly sending a user query to an LLM, the system first retrieves relevant information and provides it to the model as additional context.

In this project, the concept is applied to software repositories. Relevant source-code context can be retrieved before the LLM generates a proposed solution.

5.5 AI Agents

An AI agent is a system in which an LLM can reason about a task and interact with external tools or environments.

A simple LLM application can be represented as:

User Input → LLM → Response

An agentic system instead follows a workflow such as:

Task → Reason → Select Action → Execute Tool → Observe Result → Reason Again

This ability to interact with the environment is important for software engineering tasks.

5.6 Tool Calling

Tool calling allows an LLM-based system to invoke predefined functions to interact with an external environment.

Examples of tools relevant to software engineering include:

- Searching files
- Reading files
- Writing or modifying files
- Inspecting repository structure
- Running tests
- Examining execution errors
- Inspecting code changes

The tools provide the agent with capabilities that the language model itself does not directly possess.

5.7 Automated Testing

Testing is used as an external verification mechanism. Instead of accepting generated code solely based on the LLM's response, the system can execute tests and observe whether the modification behaves as expected.

Test results can also be used as feedback for another iteration of the agent.

6. Related Work

The design of this project was influenced by research across several areas of artificial intelligence and software engineering.

6.1 Attention Is All You Need

The Transformer architecture introduced in "Attention Is All You Need" forms the foundation of many modern language models.

I studied this work to understand the underlying architecture behind contemporary LLMs. The concept of attention is particularly important because it allows models to capture relationships between different parts of an input sequence.

This provided the foundational understanding required before studying LLM applications in software engineering.

6.2 ReAct: Reasoning and Acting

ReAct introduced an approach that combines reasoning with actions. Instead of producing a final response directly, an agent can reason about what it needs to do, perform an action using a tool, observe the result, and continue reasoning.

This concept is closely related to the architecture of my project.

The software engineering workflow can be represented as:

Task → Reason → Action → Observation → Reason → Action

For example, the agent may decide to inspect a particular file, observe the implementation, modify the file, execute a test, observe a failure, and then reconsider its previous modification.

6.3 Repository-Level Code Generation

Repository-level code generation studies the problem of generating or modifying code when information from multiple parts of a software repository may be required.

This is different from generating a small isolated function because the model needs to understand relationships between files, functions, modules, dependencies, and project conventions.

This research motivated the retrieval component of my system, where relevant repository context is selected before generating a solution.

6.4 SWE-bench

SWE-bench is an important benchmark for evaluating AI systems on real-world software engineering tasks. Instead of evaluating only isolated code-generation problems, it focuses on issues drawn from real software repositories.

Studying SWE-bench helped me understand that evaluating a coding agent should involve realistic software engineering tasks and not only subjective assessment of generated code.

It also motivated the use of test-based evaluation in my project.

6.5 LLM-Based Program Repair

Automated program repair attempts to automatically identify and correct software defects. Recent approaches increasingly use LLMs to generate candidate patches.

The general workflow is:

Bug → Candidate Patch → Test → Feedback → Improved Patch

My project follows a related idea by using the LLM to generate modifications and automated tests as feedback.

6.6 Research Gap

The literature shows that LLMs are increasingly capable of performing software engineering tasks, but several challenges remain, including repository understanding, context selection, hallucinated code, incorrect tool usage, and reliable verification.

This project therefore explores a combination of repository retrieval, agentic reasoning, tool interaction, and test-based feedback as a practical architecture for LLM-based software engineering.

7. System Architecture

The proposed system consists of several interconnected components.

7.1 High-Level Workflow

The overall workflow of the AI Software Engineering Agent is:

1. The user provides a software engineering task.
2. The agent interprets the task.
3. The repository is inspected to identify relevant information.
4. Relevant code or documentation is retrieved.
5. The retrieved context is provided to the LLM.
6. The LLM reasons about the task and determines an appropriate action.
7. The agent invokes the required software engineering tool.
8. The generated modification is applied to the repository.
9. Automated tests are executed.
10. Test results are returned to the agent.
11. If the solution fails, the agent uses the feedback to perform another iteration.
12. Once the solution satisfies the evaluation criteria, the final result is returned.

7.2 Architecture Components

The major components are:

User Interface / Task Input

Accepts the software engineering task from the user.

Agent Controller

Controls the reasoning and action loop and determines what the system should do next.

Repository Analyzer

Provides information about the repository structure and source files.

Retrieval Module

Identifies relevant repository context for the current task.

LLM

Performs natural-language reasoning and generates proposed software changes or actions.

Tool Layer

Provides functions for repository interaction and software execution.

Test and Verification Module

Executes tests and collects the resulting feedback.

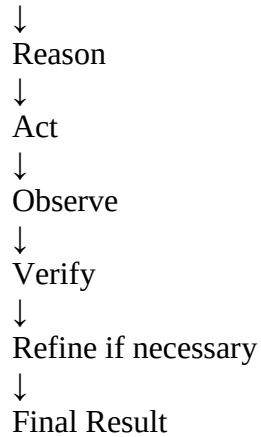
Feedback Loop

Feeds test results and execution information back into the agent for further reasoning.

7.3 Agent Loop

The central loop can be represented as:

Task
↓
Understand
↓
Retrieve



This iterative structure is the main difference between the proposed system and a simple prompt-based coding assistant.

8. Methodology

The methodology followed during the development of the project consisted of multiple stages.

Step 1: Task Understanding

The agent first receives a software engineering request expressed in natural language.

Examples include:

- Fix a bug
- Modify an existing function
- Implement a feature
- Explain an error
- Improve existing code

The task is converted into an internal representation that can guide subsequent actions.

Step 2: Repository Understanding

The agent needs to understand the environment in which the task occurs.

Repository structure, source files, test files, and other relevant artifacts are inspected.

Step 3: Context Retrieval

Rather than providing the complete repository to the LLM, relevant pieces of information are selected.

This reduces unnecessary context and allows the model to focus on code that is more closely related to the task.

Step 4: Reasoning

The retrieved context is combined with the task description and provided to the LLM.

The model determines:

- What the problem is
- Which files are relevant
- What changes may be required
- Which tool should be used next

Step 5: Tool Execution

The agent invokes appropriate tools to interact with the repository.

The tool result becomes an observation available to the next reasoning step.

Step 6: Code Modification

Based on the available context and reasoning, the agent generates a proposed code modification.

The change is applied to the relevant source file or files.

Step 7: Verification

The modified repository is tested using automated tests.

The system records whether the tests succeed or fail.

Step 8: Iterative Refinement

If tests fail, the failure information is provided to the agent.

The agent can then analyze the failure, reconsider the previous solution, and generate another modification.

This creates a feedback loop:

Generated Change → Test → Result → Agent → Improved Change

Step 9: Final Output

When the verification criteria are satisfied, the system returns the final result and relevant information about the modification.

9. Technology Stack

The project was implemented using a combination of programming, AI, and software engineering technologies.

Component	Technology
Programming Language	Python
LLM Integration	Groq API
AI Model	LLM accessed through API
Retrieval	Embedding-based retrieval
Agent Architecture	Reasoning + Tool Calling
Repository Interaction	Python-based tools
Testing	Automated test execution
Version Control	Git / GitHub
Development Environment	Local development environment

Python was selected because of its extensive ecosystem for artificial intelligence, natural language processing, automation, and software engineering tooling.

The LLM is accessed through an API rather than being trained from scratch. The focus of the project is therefore the agent architecture and software engineering workflow rather than foundation-model training.

10. Implementation

10.1 LLM Integration

The LLM is integrated into the application through an API. The model receives the task description together with relevant repository context and instructions describing the available tools and expected behavior.

The model's output is then interpreted by the agent controller.

10.2 Repository Processing

The repository is processed to identify source files and other relevant artifacts.

The system must distinguish between useful source information and irrelevant files so that unnecessary information does not dominate the model context.

10.3 Retrieval

Repository information can be divided into smaller units such as files, functions, or code chunks.

These units can be represented using embeddings and retrieved according to their semantic relevance to the current task.

The retrieved information is then supplied to the LLM as additional context.

10.4 Agent Controller

The agent controller coordinates the interaction between the LLM and external tools.

Instead of forcing the model to follow a fixed sequence for every task, the agent can determine which action is appropriate based on the current state of the problem.

10.5 Tool Layer

The tool layer provides the agent with controlled access to software engineering operations.

Examples include repository inspection, file reading, file modification, and test execution.

This allows the LLM to interact with the actual software environment.

10.6 Verification Loop

After a code modification is generated, tests are executed.

If the tests pass, the solution can proceed toward completion.

If the tests fail, the failure information is provided to the agent. The agent can then perform additional reasoning and attempt another modification.

This verification loop is one of the central components of the system.

10.7 Git-Based Change Tracking

Git can be used to inspect and track changes generated by the agent. This makes it possible to identify exactly what modifications were introduced during an agent iteration.

Git-based change inspection can also support future research involving patch quality and repository evolution.

11. Experimental Setup

The system was evaluated using software engineering tasks designed to test different aspects of the agent workflow.

The evaluation focused on whether the system could:

- Understand the given task
- Identify relevant repository information
- Generate appropriate code changes
- Successfully modify the repository
- Execute tests
- Respond to test failures
- Produce a working solution

For each task, the system recorded information about the execution process.

Important observations included:

- Whether the task was successfully completed
- Whether tests passed
- Whether the agent required multiple iterations
- Whether retrieval provided useful context
- Whether incorrect modifications were generated
- What types of failures occurred

The experimental setup was designed not only to measure successful solutions but also to study failure behavior.

12. Evaluation Metrics

The performance of the system can be evaluated using several metrics.

12.1 Task Success Rate

The percentage of tasks successfully completed by the agent.

$\text{Task Success Rate} = \text{Successful Tasks} / \text{Total Tasks}$

12.2 Test Pass Rate

The percentage of generated solutions for which the relevant tests pass.

12.3 Repair Success

For debugging tasks, repair success measures whether the generated modification resolves the reported problem without introducing additional failures.

12.4 Iterations

The number of reasoning and modification cycles required before a successful result is obtained.

A lower number of iterations can indicate a more efficient workflow, although fewer iterations do not necessarily imply a better solution.

12.5 Failure Analysis

In addition to numerical metrics, the system should be evaluated qualitatively by analyzing why failures occurred.

Possible failure categories include:

- Incorrect retrieval
- Insufficient repository context
- Incorrect reasoning
- Incorrect tool selection
- Invalid code modification
- Test failure
- Incomplete test coverage
- Context limitations

13. Results

The implemented AI Software Engineering Agent was evaluated on six software debugging tasks. Each task initially contained a failing test, and the agent was required to identify the problem, inspect the repository, modify the relevant code, and execute the tests again.

The evaluation results are summarized below.

Metric	Result
Total Bugs Evaluated	6
Successful Repairs	6
Failed Repairs	0
Repair Success Rate	100%
Cases with Failing Tests Before Repair	6/6
Cases with All Tests Passing After Repair	6/6
Average Number of Steps	5.83
Minimum Steps	5
Maximum Steps	6

Before the agent performed any modification, every evaluated bug contained at least one failing test. After the agent's modifications, all six evaluated cases had zero failing tests and returned a successful test-execution status.

The results demonstrate that the implemented agent was able to successfully resolve all six bugs included in this initial evaluation set. However, the result should be interpreted carefully because the evaluation set is small. A 100% success rate on six tasks does not establish that the system will achieve the same performance on larger or more complex repositories.

The number of agent steps ranged from 5 to 6, with an average of approximately 5.83 steps per task. The agent consistently used repository inspection, file reading, patch application, and test execution as part of the repair process.

One exception was observed in BUG-003. Although the final test result indicates a successful repair, the `agent_completed` field was recorded as false. This suggests that successful test-based repair and normal agent termination are separate aspects of the system and should be investigated further.

Overall, these initial results provide evidence that the proposed workflow can perform basic automated program repair tasks using repository inspection, code modification, and test-based verification.

14. Error Analysis

The evaluation of an LLM-based software engineering agent should consider not only successful solutions but also the reasons behind unsuccessful attempts.

Several categories of failure were observed or identified as important areas for analysis.

14.1 Retrieval Errors

The retrieval system may return code that is semantically related to the task but not sufficient to solve it. Conversely, important context may not be retrieved.

This can lead to incorrect assumptions by the LLM.

14.2 Reasoning Errors

Even when the correct code is provided as context, the LLM may incorrectly interpret the relationship between different components or misunderstand the intended behavior.

14.3 Code Generation Errors

The model may generate syntactically valid code that does not correctly implement the required behavior.

This demonstrates why generated code should not be accepted solely based on its appearance.

14.4 Tool-Use Errors

An agent may select an inappropriate tool, provide incorrect arguments, or perform an unnecessary action.

Therefore, tool selection itself is an important component of agent reliability.

14.5 Verification Errors

Passing tests provides evidence that a solution satisfies the tested behavior, but it does not prove that the software is completely correct.

If the test suite does not cover a particular edge case, an incorrect implementation may still pass.

14.6 Long-Horizon Execution

Complex tasks may require many reasoning and tool-use steps. As the number of steps increases, the probability of an intermediate error can also increase.

This is an important challenge for autonomous software engineering agents.

15. Limitations

The current implementation has several limitations.

First, the quality of the system depends on the underlying LLM. The model may generate incorrect code, misunderstand the task, or make incorrect assumptions.

Second, retrieval quality is an important limitation. If relevant repository context is not retrieved, the agent may not have sufficient information to solve the task.

Third, automated tests cannot guarantee complete correctness. A solution can pass the available tests while still containing bugs that are not covered by the test suite.

Fourth, complex repository-level tasks can require long reasoning and tool-use sequences. Errors introduced during an intermediate step can affect later stages.

Fifth, API-based LLM systems are affected by model availability, latency, token limits, and usage constraints.

Finally, the current project is primarily a prototype and research exploration rather than a production-grade autonomous software engineering platform. More extensive evaluation and comparison with established systems would be required before making claims about real-world performance.

17. Future Work

There are several directions in which this project can be extended.

17.1 Retrieval Ablation

A controlled experiment can compare:

1. LLM without repository retrieval
2. LLM with repository retrieval
3. LLM with retrieval and agentic tool use
4. LLM with retrieval, tool use, and test-based feedback

This would help determine the contribution of each component.

17.2 Improved Code Retrieval

Future work can investigate different retrieval strategies, including semantic search, hybrid lexical-semantic retrieval, function-level retrieval, dependency-aware retrieval, and repository-structure-aware retrieval.

17.3 Better Verification

Testing can be extended beyond existing unit tests using additional techniques such as static analysis, linting, type checking, security analysis, and generated tests.

17.4 Patch Quality

Future experiments can evaluate not only whether tests pass but also whether generated patches are minimal, maintainable, readable, and consistent with existing project conventions.

17.5 Repository-Level Benchmarks

The system can be evaluated on established software engineering benchmarks such as SWE-bench or other repository-level datasets.

17.6 Human-in-the-Loop Development

A future version could allow developers to review proposed changes before they are applied, creating a collaborative workflow between humans and AI agents.

17.7 Agent Trajectory Analysis

The sequence of reasoning, tool calls, observations, and failures can be recorded and analyzed. This could help identify where errors occur during long-horizon agent execution.

17.8 Reliability and Safety

Future work can investigate mechanisms for preventing destructive modifications, restricting tool permissions, validating patches, and creating automatic rollback mechanisms.

Overall, the long-term goal is to move from a prototype coding agent toward a more reliable, measurable, and research-oriented AI software engineering system.

18. Conclusion

This project explored the design and implementation of an AI Software Engineering Agent that uses a Large Language Model as part of an iterative software development workflow.

The central idea was to move beyond one-shot code generation by allowing the system to understand repository context, retrieve relevant information, interact with software engineering tools, generate code modifications, and use automated tests as feedback.

The project also provided an opportunity to connect practical implementation with current research in AI for Software Engineering. Studying Transformer-based language models, ReAct,

repository-level code generation, retrieval-augmented generation, automated program repair, and SWE-bench helped shape the architecture and evaluation approach.

One of the main lessons from the project is that the quality of an LLM-based coding system depends not only on the language model itself but also on the surrounding system. Context retrieval, tool design, verification, feedback, and evaluation all play important roles.

Although the current implementation has limitations, it provides a foundation for further research into reliable repository-level coding agents. Future work can focus on controlled experiments, retrieval strategies, stronger verification methods, benchmark-based evaluation, and analysis of agent failures.

Through this project, I gained practical experience in LLM application development while also developing a deeper interest in research at the intersection of artificial intelligence and software engineering.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. "Attention Is All You Need." Advances in Neural Information Processing Systems, 2017.
- [2] Yao, S., Zhao, J., Yu, D., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." International Conference on Learning Representations, 2023.
- [3] Zhang, et al. "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation." Research on repository-level code generation.
- [4] Jimenez, C. E., Yang, J., Wettig, A., et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" International Conference on Learning Representations, 2024.
- [5] Research literature on Large Language Models for Software Engineering and LLM-based Automated Program Repair.
- [6] Research literature on Retrieval-Augmented Generation and code retrieval.
- [7] Groq API documentation and model documentation used during implementation.
- [8] Python documentation and software testing documentation used during implementation.

Research Log

Paper 1 — Attention Is All You Need

Why I studied it:

I wanted to understand the foundation of modern LLMs before studying their applications to software engineering.

Key concept learned:

Transformer architecture and self-attention.

Connection to my project:

My software engineering agent uses an LLM, so understanding the underlying Transformer architecture helped me understand the capabilities and limitations of the model.

Paper 2 — ReAct

Why I studied it:

I wanted to understand how an LLM can move from simply generating responses to interacting with tools.

Key concept learned:

Reasoning and acting can be combined into an iterative loop.

Connection to my project:

My agent follows a similar conceptual workflow by reasoning about a task, selecting tools, observing their results, and continuing the process.

Paper 3 — Repository-Level Code Generation

Why I studied it:

I realized that software engineering tasks often require information from multiple files rather than a single code snippet.

Key concept learned:

Repository-level context and retrieval are important for code generation.

Connection to my project:

This motivated the retrieval component of my agent, which attempts to provide relevant repository information to the LLM.

Paper 4 — SWE-bench

Why I studied it:

I wanted to understand how AI systems performing software engineering tasks can be evaluated realistically.

Key concept learned:

Real-world repository issues provide a more meaningful evaluation setting than isolated coding questions.

Connection to my project:

It motivated me to think about task success, test execution, patch correctness, and failure analysis rather than evaluating generated code only by appearance.

Paper 5 — LLM-Based Program Repair

Why I studied it:

Automated program repair is closely related to the goal of generating and validating code modifications.

Key concept learned:

Generated patches should be evaluated through execution and testing.

Connection to my project:

My agent uses a feedback loop in which generated changes are tested and failures can be used to guide subsequent iterations.